In [ ]:
```
#%pip install -q pandas matplotlib scikit-learn nltk
```

In [1]:
```
import re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.feature_extraction.text import CountVectorizer, ENGLISH_STOP_WORDS
from sklearn.decomposition import LatentDirichletAllocation

CSV_PATH = "twitter_data.csv"   # Change this to your CSV file
TEXT_COLUMN = "text"            # Change this to the column containing tweets

try:
    df = pd.read_csv(CSV_PATH)
    texts = df[TEXT_COLUMN].dropna().astype(str).tolist()
    print("Loaded dataset:", len(texts), "tweets")
except (FileNotFoundError, KeyError):
    print("CSV/column not found: using a small demo dataset. Replace with real Twit
    texts = [
        "I love this phone and the camera is amazing",
        "The new update is bad and the app is slow",
        "Great service and helpful support team",
        "I hate the delays and poor customer service",
        "The product is good and works really well",
        "Terrible experience with this disappointing update",
        "The team won the match and fans are happy",
        "The match was exciting and the players were great",
        "Bad weather caused travel delays today",
        "People are discussing technology and social media"
    ]
print("Sample:", texts[:3])
```

```
CSV/column not found: using a small demo dataset. Replace with real Twitter data for
your assignment.
Sample: ['I love this phone and the camera is amazing', 'The new update is bad and t
he app is slow', 'Great service and helpful support team']
```

In [2]:
```
def clean_tweet(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\.\S+", " ", text)  # URLs
    text = re.sub(r"@\w+", " ", text)               # mentions
    text = re.sub(r"#", "", text)                    # keep hashtag word
    text = re.sub(r"[^a-z\s]", " ", text)
    tokens = [w for w in text.split() if w not in ENGLISH_STOP_WORDS and len(w) > 1
    return " ".join(tokens)

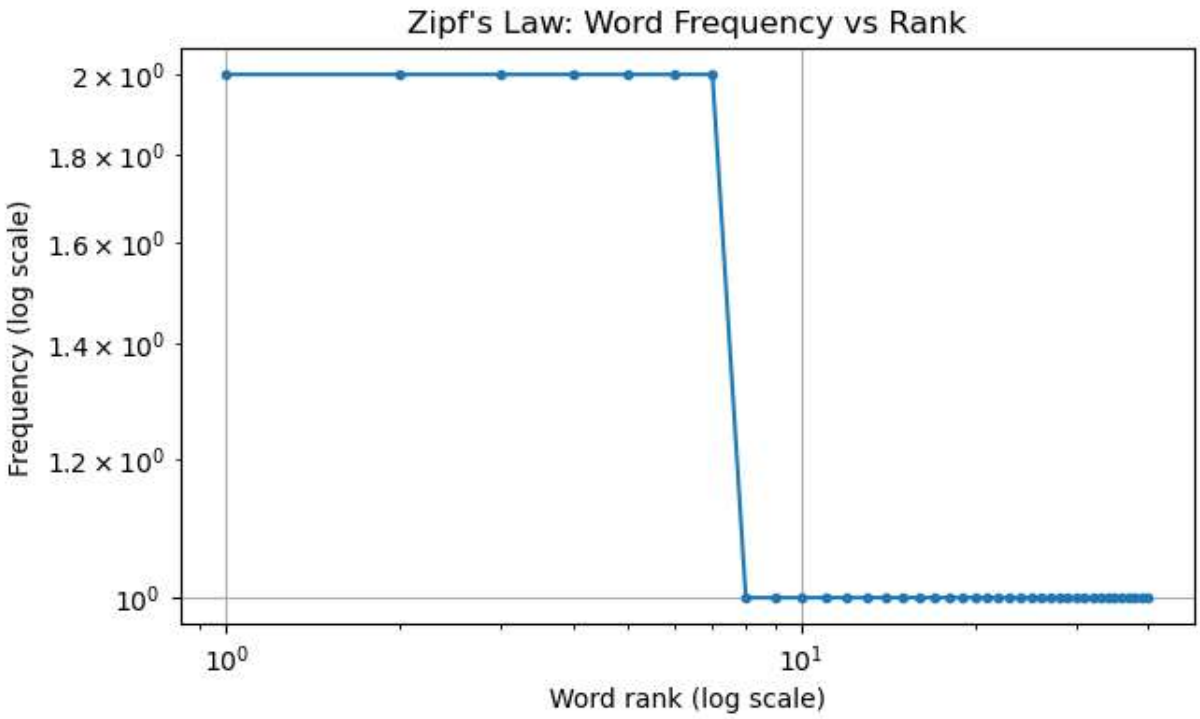
cleaned = [clean_tweet(t) for t in texts]
display(pd.DataFrame({"original": texts[:10], "cleaned": cleaned[:10]}))
```

	original	cleaned
0	I love this phone and the camera is amazing	love phone camera amazing
1	The new update is bad and the app is slow	new update bad app slow
2	Great service and helpful support team	great service helpful support team
3	I hate the delays and poor customer service	hate delays poor customer service
4	The product is good and works really well	product good works really
5	Terrible experience with this disappointing up...	terrible experience disappointing update
6	The team won the match and fans are happy	team won match fans happy
7	The match was exciting and the players were great	match exciting players great
8	Bad weather caused travel delays today	bad weather caused travel delays today
9	People are discussing technology and social media	people discussing technology social media

In [3]:
```
# Zipf's Law: word frequency against rank
all_tokens = " ".join(cleaned).split()
counts = Counter(all_tokens)
ranked = counts.most_common()
ranks = np.arange(1, len(ranked) + 1)
frequencies = np.array([count for word, count in ranked])

print("Most common words:", ranked[:15])
plt.figure(figsize=(7, 4))
plt.loglog(ranks, frequencies, marker=".")
plt.xlabel("Word rank (log scale)")
plt.ylabel("Frequency (log scale)")
plt.title("Zipf's Law: Word Frequency vs Rank")
plt.grid(True)
plt.show()
```

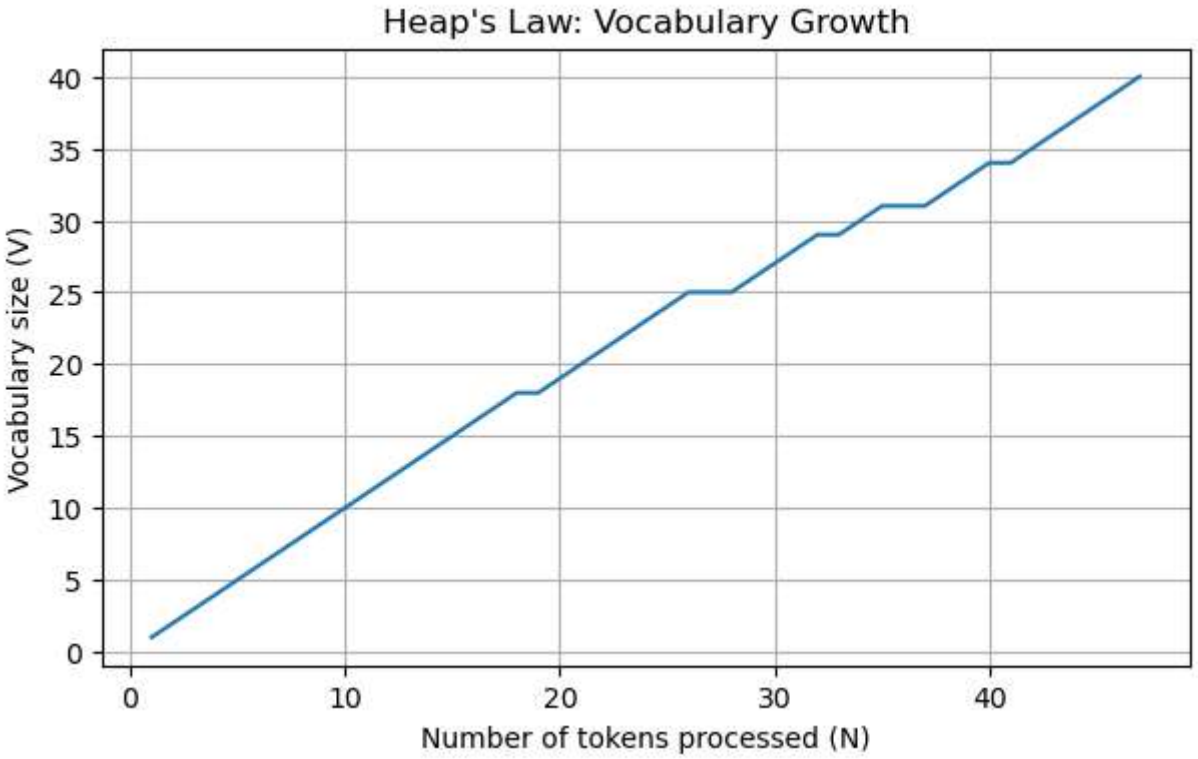
```
Most common words: [('update', 2), ('bad', 2), ('great', 2), ('service', 2), ('tea
m', 2), ('delays', 2), ('match', 2), ('love', 1), ('phone', 1), ('camera', 1), ('ama
zing', 1), ('new', 1), ('app', 1), ('slow', 1), ('helpful', 1)]
```



In [4]:
```
# Heap's Law: vocabulary size as tokens are processed
vocabulary = set()
token_counts = []
vocab_sizes = []
for i, token in enumerate(all_tokens, start=1):
    vocabulary.add(token)
    token_counts.append(i)
    vocab_sizes.append(len(vocabulary))

plt.figure(figsize=(7, 4))
plt.plot(token_counts, vocab_sizes)
plt.xlabel("Number of tokens processed (N)")
plt.ylabel("Vocabulary size (V)")
plt.title("Heap's Law: Vocabulary Growth")
plt.grid(True)
plt.show()

if len(token_counts) > 1 and vocab_sizes[-1] > 0:
    print("Final token count:", token_counts[-1], "Final vocabulary size:", vocab_s
```




```
Final token count: 47 Final vocabulary size: 40
```

In [5]:
```
# Topic identification using LDA
if len(cleaned) >= 3 and any(doc.strip() for doc in cleaned):
    vectorizer = CountVectorizer(max_df=1.0, min_df=1, stop_words="english")
    doc_term = vectorizer.fit_transform(cleaned)
    if doc_term.shape[1] >= 2:
        n_topics = min(3, doc_term.shape[0], doc_term.shape[1])
        lda = LatentDirichletAllocation(n_components=n_topics, random_state=42, max
        lda.fit(doc_term)
        words = np.array(vectorizer.get_feature_names_out())
        for topic_idx, topic in enumerate(lda.components_):
            top_words = words[topic.argsort()[-8:][::-1]]
            print(f"Topic {topic_idx + 1}: {', '.join(top_words)}")
    else:
        print("Not enough distinct words for topic modeling.")
else:
    print("Not enough non-empty documents for topic modeling.")
```

```
Topic 1: bad, delays, weather, travel, today, caused, works, product
Topic 2: service, media, discussing, technology, people, social, customer, hate
Topic 3: team, update, match, great, won, happy, fans, app
```